```
  Menu  Options  View  Utilities  Compilers  Help
 ------------------------------------------------------------------------------
 DSLIST - Data Sets Matching S990058                                  Row 1 of 9

 Command - Enter "/" to select action                 Message          Volume
 ------------------------------------------------------------------------------
          S990058                                                      *ALIAS
          S990058.COBOL.CBL                                            VPWRKC
          S990058.COBOL.DATA                                           VPWRKC
          S990058.COBOL.JCL                                            VPWRKC
          S990058.COBOL.LOAD                                           VPWRKA
          S990058.PHONE                                                VPWRKA
          S990058.S0W1.ISPF.ISPPROF                                    VPWRKD
          S990058.S0W1.SPFLOG1.LIST                                    VPWRKB
          S990058.S0W1.SPFTEMP0.CNTL                                   VPWRKA
 ***************************** End of Data Set list ***************************



 Command ===> _____________________________________________ Scroll ===> PAGE
  F1=Help    F2=Split   F3=Exit    F5=Rfind   F7=Up      F8=Down    F9=Swap
 F10=Left   F11=Right  F12=Cancel
  ONLINE                                          22,15         6:48 left
```

```
  S990058.S0W1.SPFLOG1.LIST has been kept.
 READY
LISTDS PHONE MEMBERS
 S990058.PHONE
 --RECFM-LRECL-BLKSIZE-DSORG
   FB    80    6160    PO
 --VOLUMES--
   VPWRKA
 --MEMBERS--
   PHONEADD
   PHONECMP
   PHONELST
 READY


  ONLINE                                  14,1         6:20 left
```

```
JOB PHONECMP(JOB06911) SUBMITTED
***

ONLINE                                          2,6          0:59 left
```

```
CALL COBOL.LOAD(PHONELST)
Last Name:ELLIS              First:AUSTIN             NO:904-431-8551
READY

ONLINE                                  4,1          3:16 left
```

```
 |
 READY
CALL COBOL.LOAD(PHONEADD)
 21.13.01 JOB06921 $HASP165 PHONECMP ENDED AT SVSCJES2 MAXCC=0000 CN(INTERNAL)

 Last Name ?
PIERSALL
 First Name ?
SCOTT
 Number ?
904-542-5342
 Email              ?
SCOTT@UNF.EDU
 Enter Another ?
N
 READY
CALL COBOL.LOAD(PHONELST)
 Last Name:ELLIS              First:AUSTIN            NO:904-431-8551   Email:
 Last Name:SMITH              First:JOHN              NO:904-067-6767   Email:
 Last Name:PIERSALL           First:SCOTT             NO:904-534-3424   Email:
 Last Name:CALL COBOL.LOAD(PHONFirst:SCOTT            NO:904-888-2313   Email:
 Last Name:PIERSALL           First:SCOTT             NO:904-542-5342   Email:             SCOTT@UNF.EDU
 READY
 ONLINE                                        24,1          3:16 left
```

PHONESCH:

```
      *-----------------------
      * COBOL PROGRAM TO DEMONSTRATE READING
      * A SEQUENTIAL FILE
      * RECORD-BY-RECORD DISPLAYING
      * ITS CONTENTS TO THE SCREEN.
      * Author: Austin Ellis
      * THIS PROGRAM WORKS ONLY WITH THE PHONEBOOX
      * FILE FORMAT
      *-----------------------
       IDENTIFICATION DIVISION.
       PROGRAM-ID. PHONESCH.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT PHONE-FILE ASSIGN TO PHONBOOK
              ORGANIZATION IS SEQUENTIAL.

       DATA DIVISION.
       FILE SECTION.
       FD   PHONE-FILE RECORDING MODE F
           LABEL RECORDS ARE STANDARD.
       01  PHONE-RECORD.
           05 PHONE-LAST-NAME   PIC X(20).
           05 PHONE-FIRST-NAME  PIC X(20).
           05 PHONE-NUMBER      PIC X(15).
           05 EMAIL-ADDRESS     PIC X(25).
```

```
WORKING-STORAGE SECTION.
01 LOOKUP-VALUE       PIC X(20).
01 LOOKUP-CLEAN       PIC X(20).
01 FILE-END-FLAG      PIC X VALUE "N".
01 HIT-TOTAL          PIC 9(4) VALUE 0.
01 HIT-FLAG           PIC X VALUE "N".
01 CNT-WORK           PIC 9(4) VALUE 0.

01 OUT-FIELDS.
    05 OF-LN          PIC X(20).
    05 OF-FN          PIC X(20).
    05 OF-PH          PIC X(15).
    05 OF-EM          PIC X(25).

01 TMP-IN             PIC X.

PROCEDURE DIVISION.
MAIN-SECTION.
    PERFORM READ-TERM.
    OPEN INPUT PHONE-FILE.
    PERFORM HANDLE-RECORDS UNTIL FILE-END-FLAG = "Y".
    CLOSE PHONE-FILE.
    PERFORM END-REPORT.
    STOP RUN.

READ-TERM.
    DISPLAY "Enter search term: ".
    ACCEPT LOOKUP-VALUE.
    MOVE LOOKUP-VALUE TO LOOKUP-CLEAN.

HANDLE-RECORDS.
    READ PHONE-FILE NEXT RECORD
         AT END MOVE "Y" TO FILE-END-FLAG
    END-READ
    IF FILE-END-FLAG = "Y"
         EXIT PARAGRAPH
    END-IF
    PERFORM MATCH-CHECK.

MATCH-CHECK.
    IF LOOKUP-CLEAN = SPACES
         PERFORM SHOW-LINE
         EXIT PARAGRAPH
    END-IF
```

```
        MOVE 0 TO CNT-WORK.
        INSPECT PHONE-LAST-NAME
           TALLYING CNT-WORK FOR ALL LOOKUP-CLEAN.
        IF CNT-WORK > 0
             PERFORM SHOW-LINE
             EXIT PARAGRAPH
        END-IF

        MOVE 0 TO CNT-WORK.
        INSPECT PHONE-FIRST-NAME
           TALLYING CNT-WORK FOR ALL LOOKUP-CLEAN.
        IF CNT-WORK > 0
             PERFORM SHOW-LINE
             EXIT PARAGRAPH
        END-IF

        MOVE 0 TO CNT-WORK.
        INSPECT PHONE-NUMBER
           TALLYING CNT-WORK FOR ALL LOOKUP-CLEAN.
        IF CNT-WORK > 0
             PERFORM SHOW-LINE
             EXIT PARAGRAPH
        END-IF

        MOVE 0 TO CNT-WORK.
        INSPECT EMAIL-ADDRESS
           TALLYING CNT-WORK FOR ALL LOOKUP-CLEAN.
        IF CNT-WORK > 0
             PERFORM SHOW-LINE
             EXIT PARAGRAPH
        END-IF.

    SHOW-LINE.
        MOVE PHONE-LAST-NAME   TO OF-LN.
        MOVE PHONE-FIRST-NAME  TO OF-FN.
        MOVE PHONE-NUMBER      TO OF-PH.
        MOVE EMAIL-ADDRESS     TO OF-EM.

        DISPLAY OF-LN "  "
                OF-FN "  "
                OF-PH "  "
                OF-EM.

        MOVE "Y" TO HIT-FLAG.
```

```
          ADD 1 TO HIT-TOTAL.

      END-REPORT.
          IF HIT-FLAG = "N"
               DISPLAY "NO MATCHES FOUND FOR: " LOOKUP-CLEAN
          ELSE
               DISPLAY "TOTAL MATCHES: " HIT-TOTAL
          END-IF.
```

PHONEADD:

```
*-----------------------------------------------
      *
      * COBOL Program to Demonstrate adding records
      * to a sequential file.
      *
      * The teaching tool is a simple phone book
      * that stores last name, first name, and
      * phone number.
      *
      * Author: Austin Ellis
      * ---------------------------------------------
       IDENTIFICATION DIVISION.
       PROGRAM-ID. PHONEADD.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
      * ---------------------------------------------
      * We SELECT OPTIONAL because we are
      * using OPEN EXTEND to open the phonebook file
      * and we want the file to be created if it does
      * not already exist.
      * IBM Z/OS Enterprise COBOL NOTE
      *
      * https://
      *www.ibm.com/docs/en/cobol-zos/6.3?topic=statements-open-statement
      *
      * If you want to append to a file, but are unsure
      * if the file exists, use the SELECT OPTIONAL clause
      * before opening the file in EXTEND mode. The file
      * will be created or appended to,
      * depending on whether the file exists.
      * ---------------------------------------------
          SELECT OPTIONAL PHONE-FILE ASSIGN TO PHONBOOK
```

```cobol
            ORGANIZATION IS SEQUENTIAL.

      * Recording MODE F - Fixed
      * Denotes all records in the file are the same length
      *
      * IBM z/OS ENTERPRISE COBOL NOTE
      *
      * https://
      *www.ibm.com/docs/en/cobol-zos/6.2?topic=entries-recording-mode-clause
      *
      * NOTE: THe filler field is added to:
      *     1. SUpport adding additional fields to the record
      *     2. Dataset sizes on our Mainframe default to a size of 80
      *           which is required for our record sizes
      *           using default DS Allocation in OUR z/OS Environment
       DATA DIVISION.
       FILE SECTION.
       FD  PHONE-FILE RECORDING MODE F
           LABEL RECORDS ARE STANDARD.
       01  PHONE-RECORD.
           05 PHONE-LAST-NAME   PIC X(20).
           05 PHONE-FIRST-NAME  PIC X(20).
           05 PHONE-NUMBER      PIC X(15).
           05 EMAIL-ADDRESS     PIC X(25).

       WORKING-STORAGE SECTION.
       01  PROMPT-1    PIC X(9) VALUE "Last Name".
       01  PROMPT-2    PIC X(10) VALUE "First Name".
       01  PROMPT-3    PIC X(6) VALUE "Number".
       01  PROMPT-4    PIC X(20) VALUE "Email".

       01  YES-NO      PIC X.
       01  ENTRY-OK    PIC X.

       PROCEDURE DIVISION.
       MAIN-LOGIC SECTION.
       PROGRAM-BEGIN.
           PERFORM OPENING-PROCEDURE.
           MOVE "Y" TO YES-NO.
           PERFORM ADD-RECORDS
              UNTIL YES-NO = "N".
           PERFORM CLOSING-PROCEDURE.
       PROGRAM-DONE.
           STOP RUN.
      * ------------------------------------------------------------
```

```
      * Open PHONE-FILE for APPENDING. If the file does not exist
      * it is created (due to our SELECT OPTIONAL)
      * -------------------------------------------------------
       OPENING-PROCEDURE.
           OPEN EXTEND PHONE-FILE.

       CLOSING-PROCEDURE.
           CLOSE PHONE-FILE.

       ADD-RECORDS.
           MOVE "N" to ENTRY-OK.
           PERFORM GET-FIELDS
              UNTIL ENTRY-OK = "Y".
           PERFORM ADD-THIS-RECORD.
           PERFORM GO-AGAIN.

       GET-FIELDS.
           MOVE SPACE TO PHONE-RECORD.
           DISPLAY PROMPT-1 " ? ".
           ACCEPT PHONE-LAST-NAME.
           DISPLAY PROMPT-2 " ? ".
           ACCEPT PHONE-FIRST-NAME.
           DISPLAY PROMPT-3 " ? "
           ACCEPT PHONE-NUMBER.
           DISPLAY PROMPT-4 " ? "
           ACCEPT  EMAIL-ADDRESS.
           PERFORM VALIDATE-FIELDS.

       VALIDATE-FIELDS.
           MOVE "Y" to ENTRY-OK.
           IF PHONE-LAST-NAME = SPACE
              DISPLAY "LAST NAME IS REQUIRED"
              MOVE "N" TO ENTRY-OK.
           IF PHONE-FIRST-NAME  = SPACE
              DISPLAY "FIRST NAME IS REQUIRED"
              MOVE "N" TO ENTRY-OK.
           IF PHONE-NUMBER = SPACE
              DISPLAY "PHONE NUMBER IS REQUIRED"
              MOVE "N" TO ENTRY-OK.

       ADD-THIS-RECORD.
           WRITE PHONE-RECORD.

       GO-AGAIN.
           DISPLAY "Enter Another ?".
```

```
          ACCEPT YES-NO.
          IF YES-NO = "Y"
             MOVE "Y" to YES-NO.
          IF YES-NO NOT = "Y"
             MOVE "N" to YES-NO.
```

PHONELST:

```
*-----------------------
      *
      * COBOL PROGRAM TO DEMONSTRATE READING
      * A SEQUENTIAL FILE
      * RECORD-BY-RECORD DISPLAYING
      * ITS CONTENTS TO THE SCREEN.
      * Author: Austin Ellis
      * THIS PROGRAM WORKS ONLY WITH THE PHONEBOOX
      * FILE FORMAT
      *-----------------------
       IDENTIFICATION DIVISION.
       PROGRAM-ID. PHONELST.
       ENVIRONMENT DIVISION.
       INPUT-OUTPUT SECTION.
       FILE-CONTROL.
           SELECT PHONE-FILE ASSIGN TO PHONBOOK
              ORGANIZATION IS SEQUENTIAL.

       DATA DIVISION.
       FILE SECTION.
       FD   PHONE-FILE RECORDING MODE F
           LABEL RECORDS ARE STANDARD.
       01  PHONE-RECORD.
           05 PHONE-LAST-NAME   PIC X(20).
           05 PHONE-FIRST-NAME  PIC X(20).
           05 PHONE-NUMBER      PIC X(15).
           05 EMAIL-ADDRESS     PIC X(25).

       WORKING-STORAGE SECTION.
       01 FIELDS-TO-DISPLAY.
           05 PROMPT-1           PIC X(10) VALUE "Last Name:".
           05 DISPLAY-LAST-NAME  PIC X(20).
           05 PROMPT-2           PIC X(6) VALUE "First:".
           05 DISPLAY-FIRST-NAME PIC X(20).
           05 PROMPT-3           PIC X(3) VALUE "NO:".
```

```
        05 DISPLAY-NUMBER     PIC X(15).
        05 PROMPT-4           PIC X(20) VALUE "Email:".
        05 DISPLAY-EMAIL-ADDRESS    PIC X(20).

    01 END-OF-FILE  PIC X.

    01 SCREEN-LINES PIC 99.
    01 DUMMYIN      PIC X.

    PROCEDURE DIVISION.
    MAIN-LOGIC SECTION.
    PROGRAM-BEGIN.

        PERFORM OPENING-PROCEDURE.
        MOVE ZEROES TO SCREEN-LINES.
        MOVE "N" TO END-OF-FILE.
        PERFORM READ-NEXT-RECORD.
        PERFORM DISPLAY-RECORDS
           UNTIL END-OF-FILE = "Y".
        PERFORM CLOSING-PROCEDURE.

    PROGRAM-DONE.
        STOP RUN.

    OPENING-PROCEDURE.
        OPEN INPUT PHONE-FILE.

    CLOSING-PROCEDURE.
        CLOSE PHONE-FILE.

    DISPLAY-RECORDS.
        PERFORM DISPLAY-FIELDS.
        PERFORM READ-NEXT-RECORD.

    DISPLAY-FIELDS.
        IF SCREEN-LINES = 25
           PERFORM PRESS-ENTER.
        MOVE PHONE-LAST-NAME TO DISPLAY-LAST-NAME.
        MOVE PHONE-FIRST-NAME TO DISPLAY-FIRST-NAME.
        MOVE PHONE-NUMBER TO DISPLAY-NUMBER.
        MOVE EMAIL-ADDRESS TO DISPLAY-EMAIL-ADDRESS.
        DISPLAY FIELDS-TO-DISPLAY.

   * Read the next record from the phone file
   * if we reach the end, we sent END-OF-FILE
```

```
      * to Y
       READ-NEXT-RECORD.
           READ PHONE-FILE NEXT RECORD
              AT END
              MOVE "Y" TO END-OF-FILE.

      * Prompt the user to press enter
      * Use as a simple method of screen paging
       PRESS-ENTER.
           DISPLAY "Press ENTER to continue.........".
           ACCEPT DUMMYIN.
           MOVE ZEROES TO SCREEN-LINES.
```

```
//PHONECMP   JOB  1,NOTIFY=&SYSUID
//** Compile Steps for PHONEADD and PHONELIST
//** Author: Austin Ellis
//** This JCL JOB Compiles the PHONEADD program first
//** Compilation of PHONELST proceeds only if
//** compilation of PHONEADD succeeds.
//COBRUN   EXEC IGYWCL
//**Compile and Link PHONELST Program**/
//COBOL.SYSIN  DD DSN=&SYSUID..PHONE(PHONEADD),DISP=SHR
//LKED.SYSLMOD DD DSN=&SYSUID..COBOL.LOAD(PHONEADD),DISP=SHR
//* Check the return code from the compiler
//* we only compile the next program if we get a clean
//* compile of PHONEADD
// IF RC=0 THEN
//COBRUN   EXEC IGYWCL
//COBOL.SYSIN  DD DSN=&SYSUID..PHONE(PHONELST),DISP=SHR
//LKED.SYSLMOD DD DSN=&SYSUID..COBOL.LOAD(PHONELST),DISP=SHR
// ENDIF
```

1. A Partitioned Data Set (PDS) is a mainframe storage structure that groups multiple related members within a single dataset. It works similarly to a folder that contains many individual files, each of which can be independently viewed, edited, compiled, or executed. Mainframe environments rely on PDS libraries because they offer an efficient way to manage large collections of related resources, such as COBOL programs, JCL procedures, copybooks, macros, or load modules. This structure is especially valuable in business settings where hundreds or thousands of components must be organized for development, testing, deployment, and day-to-day operations. Organizations often maintain separate PDS libraries for source

code, test JCL, and production executables, simplifying version control and the promotion of code between environments. By contrast, sequential datasets store records in a single continuous stream with no internal members. They are typically used for application data, logs, batch inputs, and other files processed from beginning to end. Sequential datasets are particularly effective for workloads such as nightly batch jobs, large data extracts, or program-owned files that benefit from straightforward, linear access.

2. The ALLOC (ALLOCATE) command in TSO/E supports numerous options that control how a dataset is created. Two options not used in this lab are LIKE() and BLKSIZE(). The LIKE() option allows a new dataset to automatically inherit all attributes—such as record format, LRECL, block size, and space parameters—from an existing dataset. This is especially useful in environments where datasets must remain uniform, such as generating daily log files that must match the format of prior logs for consistency and automated processing. The BLKSIZE() option allows the user to explicitly define the block size used to store records. Block size can have a significant impact on I/O efficiency, and many organizations tune it to match device characteristics, reduce I/O operations, or maintain compatibility with specific utilities. A practical use case for BLKSIZE() would be allocating a large transaction file that batch jobs must process quickly; choosing an optimized block size reduces the number of reads required, improving performance and lowering CPU usage. These options exist to provide system programmers and developers with fine-grained control over dataset characteristics, enabling better efficiency, compatibility, and performance across a variety of workloads.

3. The LISTA command is a shortened form of the full TSO command LISTALC, which stands for “List Allocations.” LISTALC displays all datasets currently allocated to a user’s TSO session. When combined with the HISTORY operand, it can also show allocation details such as when a dataset was created or last used. In this lab, running LISTALC HISTORY is used to verify that the dataset created in step 2 was properly allocated and cataloged. The command provides metadata that confirms the dataset’s attributes and ensures the system recognizes it. Using the full command name offers clearer documentation and aligns with traditional TSO naming conventions, where abbreviations exist for convenience but the full form remains informative and explicit.

4.

In step 2 of the lab, we used an abbreviated ALLOC command to create the sequential dataset needed by our COBOL programs. The fully expanded version removes all shorthand and uses the complete operand names. The full command is:
ALLOCATE DATASET('S990058.PHONEBOOK') NEW CATALOG SPACE(TRACKS,(1,1)) RECORDFORMAT(F) RECORDLENGTH(80)
Using the expanded form makes each parameter explicit and easier to understand, especially for new users or anyone reviewing the documentation. It clearly shows that a new dataset is being allocated, that it will be cataloged, that its space is defined in tracks rather than using shortened terms such as TRK, and that the record format and record length are fully specified. This complete version improves readability and highlights the structure and intent of the TSO command.

5. To identify the TSO level running on the system, you can use the **VERSION** command in TSO, which may also be entered in its abbreviated form, **VERS**. This command displays the installed TSO/E release and related environment details. Knowing the TSO level is important because features, command syntax, and certain system behaviors can differ between releases. This helps ensure that scripts, utilities, and documentation align with the capabilities of the specific environment. For instance, developers may need to confirm that their TSO/E version supports particular RACF functions or enhancements to dataset management.
Providing a screenshot of the command and its output verifies the system configuration and confirms that the command was executed directly on the mainframe.